Cache Penetration and Avalanche

requests for keys that don't exist, and caches that fail all at once

Phase 3 videos on YouTube

> **In one line:** cache penetration is requests for keys that don't exist passing straight through to the database; fix it with negative caching (a short-TTL "not found"), a Bloom filter of valid keys and validation and rate limits; a cache avalanche is a mass expiry or cache failure that floods the database, prevented with TTL jitter, replicas, warming and database protection.

1 Requests for things that don't exist

An attacker sends requests for random product IDs: 9,800,123, 7,415,002 and so on. None of them exist. Every request misses the cache, goes to the database, finds nothing, and **caches nothing**. The cache is useless, and the database takes every hit. That's **cache penetration**. It can be accidental too: a buggy client looping on a deleted ID, or a scraper walking an ID range.



Figure 1. Penetration: "not found" is never cached, so every request for a missing key reaches the database.

2 Fix 1: negative caching

When the database returns nothing, **cache that fact**: store a special null marker under the key with a short TTL, maybe 30 to 60 seconds. Repeat requests for the same missing key now hit the cache. Keep the TTL short (and delete the marker when the item is created), so a new item becomes visible quickly.

3 Fix 2: a Bloom filter in front

Random IDs defeat negative caching, because each ID is new. A **Bloom filter** helps: a compact bit array plus k hash functions that answers *"definitely not in the set"* or *"possibly in it"*. To add a key, set the k bits its hashes point to. To check a key, look at those bits: if any is 0, the key was never added.

Load all valid product IDs into a Bloom filter. Before touching the cache or the database, check the filter; if it says **definitely not**, reject immediately. False positives are rare and tunable; false negatives never happen. Deleting is the catch: a plain Bloom filter can't remove items, so rebuild it periodically (or use a counting or cuckoo filter).

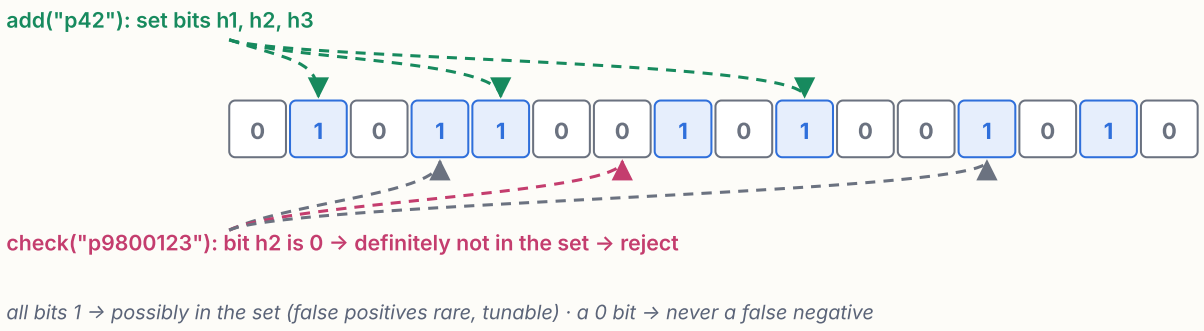


Figure 2. A Bloom filter: adding sets bits; a check that finds any 0 bit proves the key was never added.

WORKED EXAMPLE: RANDOM PRODUCT IDS

The service keeps a Bloom filter of all **10 million** valid IDs. With a 1% false-positive rate it needs about 9.6 bits per item: roughly **12 MB** of memory and 7 hash functions (RedisBloom's `BF.RESERVE products 0.01 10000000` sets this up). 99% of bogus requests are rejected in microseconds; the remaining 1% are caught by negative caching. On top: rate limiting per client, and validating the ID format at the edge.

4 Cache avalanche

An **avalanche** is when a large part of the cache fails at once and the full load falls on the database. Two common causes:

- **Mass expiry**: many keys set at the same time with the same TTL expire together (lesson 19).
- **Cache failure**: a Redis node or cluster goes down, or restarts empty.

Cause: mass expiry
keys loaded together, same TTL
→ expire together → flood of misses
fix: TTL jitter, staggered warm-up

Cause: cache failure
a Redis node or cluster dies or restarts empty
fix: replicas + failover, warm before traffic,
circuit breakers + load shedding for the DB

Figure 3. The two causes of an avalanche and how to prevent each.

Prevention: add **jitter** to TTLs; run a **highly available** cache with replicas and automatic failover (lesson 28); **warm** the cache before sending traffic to a new or restarted node; and protect the database with **circuit breakers, rate limits and load shedding** (lesson 10), so if the cache does fail, the database degrades instead of collapsing.

PENETRATION, BREAKDOWN, STAMPEDE, AVALANCHE: DON'T MIX THEM UP

- **Penetration**: keys that don't exist, every request misses.
- **Stampede / breakdown**: one hot key expires, many concurrent misses (lesson 24).
- **Avalanche**: many keys or the whole cache fail at once.

5 Key terms

Cache penetration: requests for non-existent keys reaching the DB

Negative caching: caching "not found" with a short TTL

Bloom filter: bit array: definitely not, or possibly in

False positive: filter says maybe, but the key doesn't exist

Cache avalanche: mass expiry or cache failure floods the DB

Cache warming: pre-loading hot keys before taking traffic

6 Interview questions

Q1. How do you stop requests for missing keys from hammering the database? BASIC

Cache negative results with a short TTL, put a Bloom filter of valid keys in front so impossible keys are rejected early, and add validation and rate limiting.

Q2. What is a cache avalanche? BASIC

A mass expiry or cache cluster failure that sends a flood of misses to the database, prevented with TTL jitter, replicas, cache warming and database protection.

Q3. Can a Bloom filter give false negatives? BASIC

No. If a key was added, all its bits are 1, so the filter always says 'possibly in'. It can give false positives, at a rate you choose by sizing the bit array and hash count.

Q4. Scrapers request user IDs from 1 to 100 million, most of which don't exist. What do you do? SCENARIO

Rate-limit per client and IP, reject IDs that fail a Bloom filter of existing users, cache 'not found' briefly, and consider non-sequential IDs so the space can't be walked.

Q5. Your Redis cluster restarts empty after maintenance, and the database falls over. How do you avoid that next time? SCENARIO

Use replicas with failover so a node restart doesn't empty the cache, warm hot keys before sending traffic, ramp traffic gradually, and guard the database with rate limits and load shedding.

Q6. How big is a Bloom filter for 10 million keys at 1% false positives? TRICKY

About 9.6 bits per key, so roughly 96 million bits ≈ 12 MB, with about 7 hash functions; halving the false-positive rate costs only about 1.4 more bits per key.

Next: SD-F-012 · Hot Keys backendroadmap.com